

Лабораторная работа 1 Запуск открытой модели text-to-image

Методические указания по фиксации воспроизводимого результата

Работа формирует базовую практику курса: студент запускает локально модель с открыто доступными весами, сохраняет изображение и полный манифест условий. Успех определяется не эстетическим впечатлением, а возможностью независимо восстановить запуск и объяснить границы совпадения.

Паспорт работы

Параметр	Содержание
Цель	Получить один воспроизводимый результат text-to-image и доказать происхождение каждого артефакта.
Результаты	Развернуть среду; закрепить модель и ревизию; выполнить безопасный запуск; сохранить PNG, JSON и версии; диагностировать типовую ошибку.
Компетенция	ПК-5.1 – постановка и проверка задачи применения алгоритма машинного обучения.
Связь	Лекция № 1; подготовка к ЛР № 2, где один фактор изменяется контролируемо.
Время	Аудитория 90 минут: ввод 10, среда 20, демо 30, повтор 15, анализ и защита 15. Загрузка весов и оформление отчёта допускаются вне занятия.
ПО и ресурсы	Python 3.11; PyTorch из официального установщика под устройство; Diffusers 0.40.0; Git. CUDA GPU рекомендуется; CPU допустим, но медленнее.
Модель	stabilityai/sd-turbo (https://huggingface.co/stabilityai/sd-turbo?utm_source=chatgpt.com), ревизия b261bac6fd2cf515557d5d0707481eafa0485ec2. Веса доступны без авторизации на дату проверки; условия лицензии и Acceptable Use Policy проверяются перед использованием.
Артефакты	result.png; manifest.json; environment.txt; исходный код; README с командой запуска; краткий отчёт.

Входные данные – только авторский текстовый prompt без персональных данных, закрытых брифов, товарных знаков и реальных лиц. Термин «открытая модель» здесь означает

доступность весов и документации для локального исследования; он не отменяет лицензионных условий и не утверждает соответствие лицензии критериям OSI.

Теоретический минимум

Text-to-image pipeline получает prompt, преобразует текст в представление и итеративно превращает шум в латентное изображение, которое декодируется в пиксели. Для SD Turbo разработчик рекомендует 1–4 шага, размер 512×512 и guidance_scale=0.0; отрицательный prompt и обычный classifier-free guidance в этой модели не используются.

Контрольная сумма SHA-256 идентифицирует байты файла. Совпадение SHA-256 в повторном запуске той же зафиксированной среды – сильный тест точного совпадения артефакта. Несовпадение между разными устройствами не доказывает ошибку: сначала сравнивают окружение, dtype и версии, затем визуальный и содержательный результат.

Поле	Зачем фиксировать	Граница интерпретации
model_id и revision	Определяют конкретные веса и конфигурацию.	Одинаковое имя без ревизии может указывать на обновлённый снимок.
prompt	Фиксирует условие генерации.	Пунктуация и порядок слов могут влиять на результат.
seed	Восстанавливает начальное псевдослучайное состояние.	Не компенсирует различия платформ и библиотек.
steps, guidance, size	Определяют режим инференса.	Допустимые диапазоны зависят от модели.
device, dtype, packages	Описывают вычислительную среду.	Полное совпадение не всегда гарантирует побитовое равенство.
SHA-256	Проверяет целостность конкретного файла.	Не измеряет эстетическое качество.

Подготовка среды

```
mkdir ai-creative-lab01
cd ai-creative-lab01
python -m venv .venv
# Linux/macOS
source .venv/bin/activate
# Windows PowerShell: .venv\Scripts\Activate.ps1

python -m pip install --upgrade pip
# Сначала установите PyTorch командой с https://pytorch.org/get-started/locally/
python -m pip install "diffusers==0.40.0" "transformers>=4.51,<5"
"accelerate>=1.2,<2" "safetensors>=0.5,<1" "Pillow>=11,<13"
mkdir -p data src configs artifacts reports
```

```
python -m pip freeze > reports/environment.txt
```

Для Windows создайте папки вручную или командами PowerShell. Смысл структуры одинаков: data – описание входа; src – код; configs – параметры; artifacts – результаты; reports – окружение и отчёт. Не включайте кэш весов в сдаваемый архив.

Проверка среды до загрузки весов

```
python -c "import torch, diffusers;
print(torch.__version__);
print(diffusers.__version__);
print(torch.cuda.is_available())"
```

Наблюдаемый результат: печатаются версии; третья строка сообщает True при доступной CUDA. Это проверка окружения, а не ожидаемое заранее числовое значение.

Полный демонстрационный пример

Мини-задача: создать квадратный концепт афиши научного фестиваля без текста, логотипов и людей. Критерий успешности: программа завершается без исключения и создаёт PNG 512×512, JSON-манифест со всеми параметрами и SHA-256. Визуальное соответствие брифу оценивается отдельно.

Сохраните следующий код как src/generate_once.py.

```
from pathlib import Path
from datetime import datetime, timezone
import hashlib, importlib.metadata as md, json, platform, time
import torch
from diffusers import AutoPipelineForText2Image

MODEL_ID = "stabilityai/sd-turbo"
REVISION = "b261bac6fd2cf515557d5d0707481eafa0485ec2"
PROMPT = (
    "editorial poster concept for a local science festival, "
    "abstract geometric light installation, deep blue and amber palette, "
    "clean composition, no text, no logo, no people"
)
SEED, STEPS, GUIDANCE, SIZE = 20260917, 1, 0.0, 512
out = Path("artifacts/run_001")
out.mkdir(parents=True, exist_ok=True)

device = "cuda" if torch.cuda.is_available() else "cpu"
dtype = torch.float16 if device == "cuda" else torch.float32
load_args = dict(revision=REVISION, use_safetensors=True, torch_dtype=dtype)
if device == "cuda":
    load_args["variant"] = "fp16"

started = time.perf_counter()
pipe = AutoPipelineForText2Image.from_pretrained(MODEL_ID, **load_args)
pipe = pipe.to(device)
generator = torch.Generator(device="cpu").manual_seed(SEED)
image = pipe(
    prompt=PROMPT,
    num_inference_steps=STEPS,
    guidance_scale=GUIDANCE,
    height=SIZE,
    width=SIZE,
```

```

generator=generator,
).images[0]
elapsed = time.perf_counter() - started

image_path = out / "result.png"
image.save(image_path)
sha256 = hashlib.sha256(image_path.read_bytes()).hexdigest()
packages = {name: md.version(name) for name in
             ["torch", "diffusers", "transformers", "accelerate", "safetensors"]}
manifest = {
    "created_utc": datetime.now(timezone.utc).isoformat(),
    "model_id": MODEL_ID, "revision": REVISION,
    "prompt": PROMPT, "seed": SEED, "steps": STEPS,
    "guidance_scale": GUIDANCE, "height": SIZE, "width": SIZE,
    "device": device, "dtype": str(dtype),
    "platform": platform.platform(), "packages": packages,
    "elapsed_seconds_measured": elapsed,
    "artifact": str(image_path), "sha256": sha256,
}
(out / "manifest.json").write_text(
    json.dumps(manifest, ensure_ascii=False, indent=2, encoding="utf-8")
)
print(json.dumps(manifest, ensure_ascii=False, indent=2))

python src/generate_once.py | tee reports/run_001.log

```

Ожидаемые промежуточные и итоговые результаты

- При первом запуске загружается закреплённый снимок модели; объём и время зависят от кэша и канала.
- В artifacts/run_001 появляется result.png размером 512×512 и manifest.json.
- В stdout и reports/run_001.log отображаются фактически измеренное время, среда и 64-символьная строка SHA-256. Конкретные значения заранее не задаются.
- Повторный запуск в тех же условиях используется для сравнения SHA-256; различия расследуются, а не скрываются.

Проверка артефактов

```

python -c "from PIL import Image;
im=Image.open('artifacts/run_001/result.png'); print(im.size, im.mode)"
python -m json.tool artifacts/run_001/manifest.json

```

Наблюдаемый результат первой команды — размер (512, 512) и цветовой режим, обычно RGB. Вторая команда должна вывести валидный JSON. Фактический режим фиксируется в отчёте, а не подменяется ожидаемым.

Намеренно введённая типовая ошибка

Ошибка: заменить `generator=torch.Generator(device="cpu").manual_seed(SEED)` на `generator=torch.Generator(device="cuda").manual_seed(SEED)` и затем повторить запуск на CPU-машине.

Симптом: исключение о недоступном CUDA или несовместимом устройстве генератора. Диагностика: вывести `device` и `torch.cuda.is_available()`, проверить строку создания `Generator`.

Исправление: создать CPU Generator, как в эталонном коде, либо использовать устройство только после проверки доступности. После исправления удалить неполный каталог запуска и выполнить команду заново.

Ограничение демо: совпадение SHA-256 ожидается прежде всего внутри одной зафиксированной среды. Официальная документация PyTorch прямо предупреждает, что полная воспроизводимость между релизами и платформами не гарантируется. Вопрос для обсуждения: когда визуально эквивалентный, но побитово другой результат приемлем для медиапроизводства?

Задание студенту

1. Создайте структуру проекта и зафиксируйте environment.txt. Контрольная точка: версии читаются, секреты и кэш весов отсутствуют.
2. Возьмите свой вариант и сформулируйте безопасный prompt с предметом, композицией, палитрой и ограничениями. Контрольная точка: нет реальных лиц, логотипов и закрытых данных.
3. Запустите модель с закреплённой ревизией. Контрольная точка: PNG и JSON созданы одним запуском.
4. Повторите запуск без изменения условий. Контрольная точка: сравните SHA-256; при расхождении зафиксируйте диагностику.
5. Воспроизведите типовую ошибку на безопасной копии кода, опишите симптом и исправление.
6. Соберите отчёт и архив без виртуального окружения и кэша модели.

Чек лист самопроверки

- ☐ [] Указаны model_id и полная revision.
- ☐ [] Сохранены prompt, seed, steps, guidance_scale, размер, device и dtype.
- ☐ [] environment.txt создан после установки.
- ☐ [] PNG открывается, JSON проходит json.tool, SHA-256 имеет 64 шестнадцатеричных символа.
- ☐ [] Результат не выдан за документальный факт; лицензия и ограничения упомянуты.
- ☐ [] Команда запуска повторяется из чистой папки проекта.

Индивидуальные варианты

Вариант 1

Контекст: Афиша фестиваля робототехники

Изменяемый фактор или решение: структура prompt: объект, среда, палитра, композиция

Обязательная проверка: один запуск и точный повтор

Артефакт: концепт афиши и манифест

Критерий результата: воспроизводимость и соответствие четырём элементам prompt

Ограничение или риск: не использовать логотипы производителей

Вариант 2

Контекст: Обложка подкаста о городской среде

Изменяемый фактор или решение: выбор визуальной метафоры

Обязательная проверка: сохранить запуск и проверить SHA-256

Артефакт: квадратная обложка и журнал

Критерий результата: метафора считывается без текста

Ограничение или риск: не изображать узнаваемых людей

Вариант 3

Контекст: Концепт постера научной выставки

Изменяемый фактор или решение: ограничение «по text» и чистая композиция

Обязательная проверка: повторить запуск без изменения конфигурации

Артефакт: PNG, JSON, таблица проверки

Критерий результата: нет имитации читаемого текста

Ограничение или риск: не выдавать образ за научную визуализацию

Вариант 4

Контекст: Визуал для экологического лектория

Изменяемый фактор или решение: палитра и тип освещения

Обязательная проверка: проверить размер и контрольную сумму

Артефакт: изображение и манифест

Критерий результата: палитра соответствует брифу

Ограничение или риск: избегать ложных экологических утверждений

Вариант 5

Контекст: Обложка электронного музыкального релиза

Изменяемый фактор или решение: описание материала и фактуры

Обязательная проверка: два идентичных запуска

Артефакт: обложка и README

Критерий результата: повторяемость и отсутствие чужих знаков

Ограничение или риск: не имитировать конкретного живущего художника

Вариант 6

Контекст: Постер для медиалаборатории

Изменяемый фактор или решение: иерархия объектов в prompt

Обязательная проверка: валидировать JSON и PNG

Артефакт: постер и журнал

Критерий результата: главный объект визуально доминирует

Ограничение или риск: не использовать клиентские материалы

Вариант 7

Контекст: Иллюстрация к статье о квантовых вычислениях

Изменяемый фактор или решение: абстрактность вместо псевдодокументальности

Обязательная проверка: повторить и сравнить SHA-256

Артефакт: иллюстрация и карточка ограничений

Критерий результата: явно обозначена концептуальность

Ограничение или риск: не представлять изображение как схему устройства

Вариант 8

Контекст: Концепт заставки для образовательного видео

Изменяемый фактор или решение: безопасная зона центра композиции

Обязательная проверка: проверить единый конфиг

Артефакт: кадр и манифест

Критерий результата: центр остаётся пригоден для последующего титра

Ограничение или риск: текст добавляется только в редакторе

Вариант 9

Контекст: Обложка каталога цифрового искусства

Изменяемый фактор или решение: контраст фона и центрального объекта

Обязательная проверка: зафиксировать запуск и повтор

Артефакт: обложка и журнал

Критерий результата: объект отделён от фона

Ограничение или риск: не копировать узнаваемую работу

Вариант 10

Контекст: Афиша дня открытых дверей

Изменяемый фактор или решение: дружелюбное настроение без лиц

Обязательная проверка: проверить метаданные среды

Артефакт: афиша и environment.txt

Критерий результата: настроение поддерживает аудиторию

Ограничение или риск: избегать возрастных стереотипов

Вариант 11

Контекст: Иллюстрация для материала о кибербезопасности

Изменяемый фактор или решение: метафора защиты без интерфейсных клише

Обязательная проверка: запустить и провалидировать артефакты

Артефакт: иллюстрация и JSON

Критерий результата: метафора объяснена в отчёте

Ограничение или риск: не показывать реальные секреты и ключи

Вариант 12

Контекст: Концепт упаковки медианабора

Изменяемый фактор или решение: форма и материал объекта

Обязательная проверка: повторить запуск

Артефакт: рендер-концепт и манифест

Критерий результата: объект имеет различимую форму

Ограничение или риск: не использовать чужую айдентику

Вариант 13

Контекст: Обложка серии лекций об ИИ

Изменяемый фактор или решение: визуализация взаимодействия человека и системы

Обязательная проверка: сравнить два точных повтора

Артефакт: обложка и протокол

Критерий результата: человек не представлен подчинённым автоматом

Ограничение или риск: избегать антропоморфизации модели

Вариант 14

Контекст: Постер фестиваля света

Изменяемый фактор или решение: цветовой контраст

Обязательная проверка: проверить 512×512 и SHA-256

Артефакт: постер и журнал

Критерий результата: контраст заметен и описан

Ограничение или риск: учесть риск мерцающих паттернов при анимации

Вариант 15

Контекст: Иллюстрация к статье о цифровых архивах

Изменяемый фактор или решение: метафора поиска и сохранения

Обязательная проверка: валидировать JSON

Артефакт: иллюстрация и README

Критерий результата: метафора связана с темой

Ограничение или риск: не показывать персональные документы

Вариант 16

Контекст: Концепт сцены для интерактивной инсталляции

Изменяемый фактор или решение: масштаб и точка обзора

Обязательная проверка: один запуск и повтор

Артефакт: концепт-арт и манифест

Критерий результата: масштаб читается по нейтральным объектам

Ограничение или риск: без изображения реальной площадки

Вариант 17

Контекст: Обложка сборника о будущем города

Изменяемый фактор или решение: баланс утопических и реалистичных деталей

Обязательная проверка: проверить полную конфигурацию

Артефакт: обложка и журнал

Критерий результата: вывод не утверждает прогноз как факт

Ограничение или риск: избегать политической символики

Вариант 18

Контекст: Постер студенческого хакатона

Изменяемый фактор или решение: динамичная композиция без брендов

Обязательная проверка: повторить запуск

Артефакт: постер и JSON

Критерий результата: динамика объяснена формой и светом

Ограничение или риск: не использовать логотипы платформ

Вариант 19

Контекст: Иллюстрация для курса по визуализации данных

Изменяемый фактор или решение: абстрактный образ данных

Обязательная проверка: проверить целостность файлов

Артефакт: иллюстрация и манифест

Критерий результата: образ не маскируется под реальный график

Ограничение или риск: не выдумывать статистику

Вариант 20

Контекст: Концепт титульного кадра виртуальной выставки

Изменяемый фактор или решение: глубина пространства и пустая зона навигации

Обязательная проверка: точно повторить запуск

Артефакт: кадр и протокол

Критерий результата: есть пригодная зона для интерфейса

Ограничение или риск: не включать читаемый текст в генерацию

Структура отчёта и защита

1. Титульный лист и номер варианта.
2. Цель, постановка задачи и критерии успешности.
3. Происхождение prompt, ограничения данных и лицензии.
4. Среда, версии, модель, revision, устройство и параметры.

5. Ход работы, команда запуска и существенный код.
6. PNG, JSON, SHA-256 и результат повтора.
7. Типовая ошибка, диагностика, ограничения и угрозы валидности.
8. Вывод по цели.
9. Ссылка на архив или репозиторий и индивидуальный вклад.

Контрольные вопросы

1. Чем модель отличается от pipeline и пользовательского приложения?
2. Почему имя модели без revision недостаточно?
3. Что фиксирует seed и чего он не фиксирует?
4. Почему CPU Generator полезен для сопоставимости в Diffusers?
5. Что доказывает совпадение SHA-256?
6. Почему различие SHA-256 между CPU и GPU не обязательно означает ошибку?
7. Зачем сохранять dtype и версии библиотек?
8. Какие лицензионные и конфиденциальные ограничения применялись в вашем варианте?
9. Как вы диагностировали намеренную ошибку?
10. Какой вывод о пригодности результата можно сделать и какой нельзя?

Оценивание

Блок	Базовый уровень	Средний уровень	Продвинутый уровень	Вес
Артефакт и воспроизводимость	Файлы есть, запуск повторяется в той же среде.	Манифест полный, повторный запуск сопоставим.	Есть контроль целостности и анализ границ воспроизводимости.	35 %
Постановка, данные и методика	Цель и параметры названы.	Фактор отделён от контролируемых переменных.	Методика обоснована и позволяет независимую проверку.	25 %
Анализ, ошибки и ограничения	Описан результат.	Разобраны ошибка и минимум два ограничения.	Сделаны проверяемые выводы, альтернативные объяснения и риск-анализ.	25 %
Отчёт и коммуникация	Структура соблюдена, ответы понятны.	Артефакты связаны с выводами.	Защита краткая, доказательная, термины используются точно.	15 %

Критические ошибки: отсутствуют исходный prompt или seed; модель или ревизия не указаны; результат заменён чужим изображением; не сохранён журнал среды; персональные либо нелицензированные материалы загружены без основания; вывод не следует из представленных запусков. При любой из этих ошибок работа не считается воспроизводимой до исправления.

Источники

- Карточка модели stabilityai/sd-turbo. Проверено 17 сентября 2026 года.
<https://huggingface.co/stabilityai/sd-turbo>

- Diffusers Reproducibility. Проверено 17 сентября 2026 года.
https://huggingface.co/docs/diffusers/main/en/using-diffusers/reusing_seeds
- PyTorch Reproducibility. Проверено 17 сентября 2026 года.
<https://docs.pytorch.org/docs/stable/notes/randomness.html>
- Diffusers 0.40.0 Installation и AutoPipeline API. Проверено 17 сентября 2026 года.
<https://huggingface.co/docs/diffusers/v0.40.0/en/installation>